

Algorithm Documentation: Triangle Inequality Problem

1. Overview

The goal of the project is to determine if any three elements in a given array can form a valid triangle. According to the Triangle Inequality Theorem, three sides a , b , and c form a triangle if and only if:

$a + b > c$ AND $a + c > b$ AND $b + c > a$

2. Brute Force Approach (Non-Recursive)

2.1 Pseudo-code

```
FUNCTION isTriangle(a, b, c):
    IF (a + b > c) AND (a + c > b) AND (b + c > a):
        RETURN 1
    ELSE:
        RETURN 0

FUNCTION bruteForce(arr[], n):
    FOR i FROM 0 TO n-1:
        FOR j FROM i+1 TO n-1:
            FOR k FROM j+1 TO n-1:
                IF isTriangle(arr[i], arr[j], arr[k]):
                    RETURN 1
    RETURN 0
```

2.2 Analysis & Time Complexity

The brute force algorithm employs three nested loops to check every possible combination of triplets in the array.

Component	Big-O	Reason
Outer loop (i)	$O(n)$	Iterates from 0 to $n-3$
Middle loop (j)	$O(n)$	Iterates from $i+1$ to $n-2$
Inner loop (k)	$O(n)$	Iterates from $j+1$ to $n-1$
Overall Time	$O(n^3)$	Triple nested loops: $n*(n-1)*(n-2)/6$

3. Optimized Approach (Sorting + Recursive)

3.1 Pseudo-code

```
Algorithm OptimisedSolutionRecursive(arr[], n):
```

```
    MergeSort(arr, 0, n - 1)
```

```
    RETURN RecursiveSolution(arr, n, 0)
```

```
Algorithm RecursiveSolution(arr[], n, idx):
```

```
    IF idx + 2 >= n: RETURN FALSE
```

```
    IF isTriangle(arr[idx], arr[idx+1], arr[idx+2]):
```

```
        RETURN TRUE
```

```
    RETURN RecursiveSolution(arr, n, idx + 1)
```

3.2 Analysis & Time Complexity

By sorting the array first, we only need to check adjacent triplets. If $A[idx] + A[idx+1] > A[idx+2]$, then the other two triangle conditions are automatically satisfied because the array is sorted.

- Time Complexity Break-down:
 - Sorting: $O(n \log n)$ using Merge Sort.
 - Checking: $O(n)$ for the recursive scan of adjacent triplets.
 - Total Time: $O(n \log n)$ as it is the dominant term.

4. Comparison Summary

Feature	Brute Force	Optimized (Sorting)
Strategy	Checks all triplets	Sorts then checks adjacent
Time Complexity	$O(n^3)$	$O(n \log n)$
Implementation	Iterative	Recursive
Efficiency	Slow for large n	Highly efficient